Framing Sections	Description and commitments
1. Project Identity & Repository	**Project:** BikeFlow AI - Predictive Bike-Share Rebalancing Platform **Student:** El Asraoui Abdelmoghit **Repository:** https://github.com/abdelmoghit-elasraoui/BikeFlow-AI
2. Existing Baseline Summary (Not Applicable)	Not applicable. BikeFlow AI is a new S5 project selected and initiated from scratch; it is not an evolution or fork of a previous S4/second-year application. Development therefore starts directly from the new project scope defined in this roadmap.
3. Target Audience & Potential Client	**Primary user:** a rebalancing dispatcher or operations supervisor at a docked bike-share operator who must maintain bike and dock availability across many stations with limited field crews. **Potential client / pilot:** operators using the GBFS standard; Citi Bike is the pilot data source because it provides public historical trip files and a live GBFS feed. The architecture remains transferable to other GBFS-compliant systems.
4. Problem Statement & Client Need	Bike demand is strongly imbalanced across locations and time. During peaks, some stations become empty while others become full, making the service unusable until staff rebalance bikes. Reactive dispatch means operators often intervene after the shortage has already occurred, while limited crews cannot inspect every station continuously. The client needs an early-warning system that identifies which stations are likely to fail in the next 30-60 minutes and ranks where intervention is most urgent.
5. Innovation Vision (S5)	BikeFlow AI transforms a descriptive station-monitoring dashboard into a proactive rebalancing decision-support system. It predicts short-term arrivals and departures, combines them with the live number of bikes/docks, and estimates the risk that a station will become empty or full before the failure occurs. Instead of simply showing current availability, the system produces a ranked rebalancing queue with predicted shortage/surplus and a suggested number of bikes to add or remove. The model is continuously evaluated against newly observed station states so the system can detect degradation and adapt when mobility patterns change.

Framing Sections	Description and commitments
6. Artificial Intelligence Component	**Predictive Machine Learning - spatiotemporal demand forecasting.** Two supervised models estimate the number of departures and arrivals for each station over the next 30-60 minutes using recent trip flows, temporal features, station characteristics, calendar/holiday context, weather forecasts and the live GBFS station state. The predicted flows are converted into empty/full risk and operational priority. Once enough live GBFS history has been archived, a direct shortage-risk classifier can be evaluated as a second-stage model. Workflow: Data ingestion -> feature generation -> demand forecast -> shortage-risk score -> ranked rebalancing queue -> actual future station state -> model evaluation.
7. Local AI Stack (Excluding APIs)	**Python, Pandas and NumPy** for preprocessing and station-time feature engineering. **Scikit-learn** for historical-average/Poisson or regression baselines and evaluation. **LightGBM or XGBoost** for the main CPU-friendly forecasting models. **SHAP** may be used to expose the main factors behind high-risk forecasts. Training data: recent historical Citi Bike trip records aggregated by station and time window, joined with calendar features and Open-Meteo historical forecast/weather variables. Inference uses the live GBFS feed. No black-box AI API is required.
8. Second Engineering Domain	**Cloud & DevOps (MLOps).** The innovation in this domain is not cloud hosting itself; it is the automated model-lifecycle loop: persist predictions before the horizon, compare them with the later GBFS observations, monitor forecast error and data drift, trigger controlled retraining, and promote a candidate model only if it improves the validation gate. **Technologies:** MLflow for experiment/model versioning, Evidently (or equivalent statistical checks) for drift/quality monitoring, Apache Airflow for scheduled ingestion/evaluation/retraining, Docker and GitHub Actions for reproducibility/CI-CD, and AWS S3/PostgreSQL/EC2 (or equivalent) for data, services and public deployment.

Framing Sections	Description and commitments
9. Sprint 1: Business Logic & AI (Session 6)	Build the validated data and AI backend. Download a recent 12-24 month slice of historical trip data, normalize stations, aggregate rides into station x 30-minute arrival/departure targets, and join calendar plus weather features. Start archiving the live GBFS station feed for future ground truth. Implement a historical-average baseline and the LightGBM/XGBoost forecasting models with chronological validation. Expose FastAPI endpoints for current station state, demand forecast and risk ranking. Store predictions with their forecast horizon so they can later be scored against reality.
10. Sprint 2: Integration & Frontend (Session 7)	Integrate the innovation into an operator-facing web dashboard. Display a live station map with normal / warning / critical risk levels, current bikes/docks, 30- and 60-minute forecasts, and a ranked rebalancing priority list. A station detail view will show recent flow, predicted arrivals/departures, estimated shortage/surplus and the recommended action (add/remove bikes). The interface will refresh from the FastAPI backend without requiring manual dataset uploads.
11. Sprint 3: Robustness & Security (Session 9)	Apply peer-review fixes and harden the full data-to-prediction workflow. Validate external payload schemas and timestamps, detect stale/missing GBFS or weather data, handle station closures/capacity changes, and fall back to a historical baseline when live features or the model are unavailable. Implement JWT/RBAC for operator/admin routes, secret management, rate limiting and input validation. Add prediction-quality monitoring, drift checks, automated tests for data leakage and edge cases, and alerts when recent model error exceeds the accepted threshold.
12. Sprint 4: Cloud & Industrialization (Sessions 10-11)	Finalize Docker packaging for the web application, FastAPI service, PostgreSQL, Airflow and MLflow components. Configure a GitHub Actions pipeline for tests, image build and deployment, then publish the application on AWS/Render or an equivalent cloud environment. Schedule live GBFS ingestion and model evaluation in the cloud, persist raw snapshots/model artifacts in object storage, and demonstrate the MLOps gate: recent prediction error/drift -> candidate retraining -> MLflow comparison -> promote only if the candidate improves the defined metric.